UADE
Proceso de desarrollo de Software
Actividad 2
Docente: Moisés Evaristo Bueno
Grupo: 06

Actividad 2 - Ferretería Bulón

1. Introducción

Este informe complementa el relevamiento presentado en la Entrega 1 y da respuesta a la Actividad 2: presentar la estructura general del sistema de gestión de stock de Bulón S.R.L. y evaluar los patrones de diseño que se aplicarán durante el desarrollo. Se toma como base la arquitectura y tecnologías definidas previamente (React/TypeScript en el frontend, Java con Spring Boot en el backend, y SQL Server como base de datos, organizados como un monolito modular) y se proponen seis patrones de diseño orientados a resolver problemas concretos del dominio: gestión de productos, movimientos de stock, alertas de reposición y generación de reportes.

2. Estructura general del sistema

El sistema se organiza como una aplicación cliente-servidor con arquitectura en capas dentro de un backend monolítico modular, evitando la complejidad de microservicios en esta primera versión. El frontend en React y TypeScript consume una API REST expuesta por el backend en Java/Spring Boot, el cual separa responsabilidades en capas de Controller, Service y Repository, y organiza el dominio en módulos independientes (Productos, Movimientos de Stock, Proveedores, Usuarios y Roles). La persistencia se resuelve contra SQL Server mediante JPA.

3. Patrones de diseño evaluados

Se evaluaron y seleccionaron seis patrones de diseño (superando el mínimo de cinco solicitado), aplicando principios SOLID y GRASP para asignar responsabilidades de forma cohesiva y con bajo acoplamiento, tal como se trabajó en clase. Para cada patrón se indica el problema que resuelve, dónde se aplicaría en el sistema de Bulón y su representación en un diagrama de clases simplificado.

3.1 Singleton — conexión a la base de datos

Problema: el acceso a SQL Server debe realizarse a través de una única instancia de configuración/pool de conexiones, evitando crear conexiones redundantes y centralizando su configuración.

Aplicación: una clase DatabaseConnectionManager expone un método estático getInstance() que siempre devuelve la misma instancia; los distintos Repository de cada módulo (Producto, Movimiento, Proveedor) la reutilizan en lugar de instanciar su propia conexión.

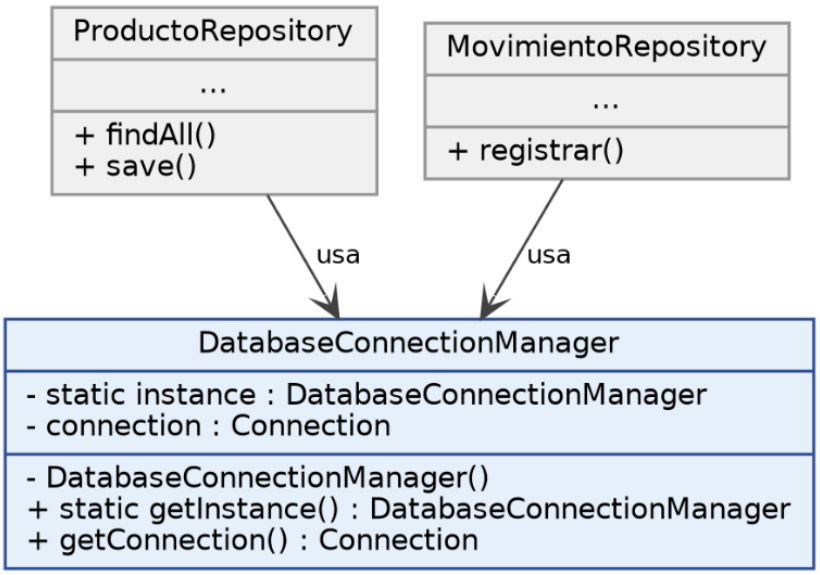


Figura 2. Singleton aplicado a la conexión con la base de datos.

3.2 Factory Method — creación de movimientos de stock

Problema: el sistema debe crear distintos tipos de movimiento (ingreso, salida, devolución, rotura, ajuste, traslado) sin acoplar el servicio que los registra a cada clase concreta mediante múltiples condicionales.

Aplicación: una clase abstracta MovimientoStockCreator define el método fábrica crearMovimiento(), y cada subclase concreta (IngresoCreator, SalidaCreator, TrasladoCreator, etc.) decide qué objeto MovimientoStock instanciar. Esto respeta el **principio Open/Closed**: agregar un nuevo tipo de movimiento no requiere modificar el código existente.

3.3 Strategy — validación y ejecución según tipo de movimiento

Problema: cada tipo de movimiento actualiza el stock de forma distinta (un ingreso suma cantidades, una salida resta y valida contra el stock mínimo, un traslado mueve cantidades entre sucursal y depósito).

Aplicación: una interfaz MovimientoStrategy define validar() y actualizarStock(), implementada por IngresoStrategy, SalidaStrategy y TrasladoStrategy, entre otras. El MovimientoStockService delega en la estrategia correspondiente según el tipo de movimiento recibido, en lugar de concentrar toda la lógica en un único método con condicionales.

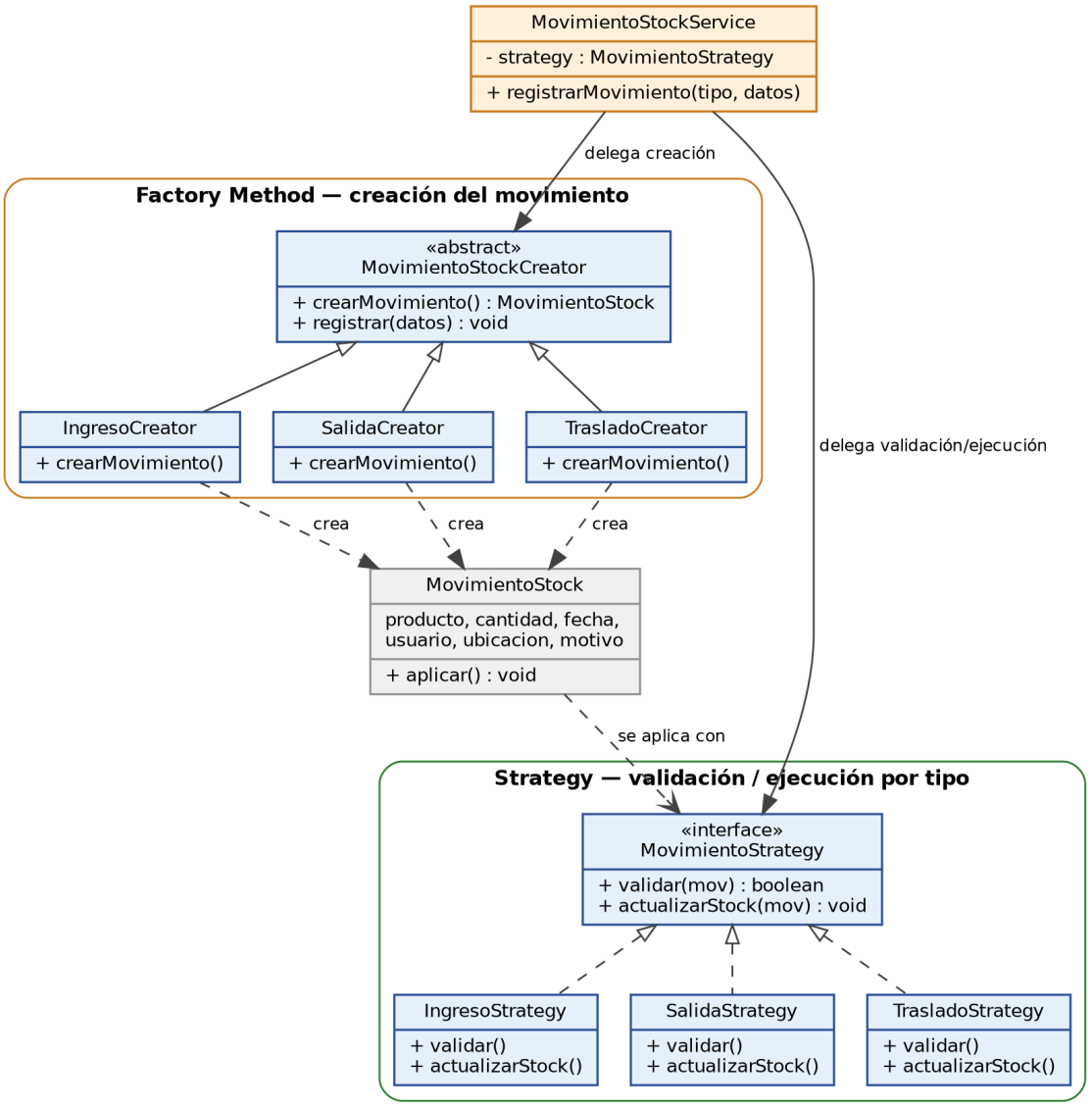


Figura 3. Factory Method (creación) y Strategy (validación/ejecución) para los movimientos de stock.

3.4 Observer — alertas de stock mínimo

Problema: cuando el stock de un producto alcanza el mínimo configurado, distintos interesados (encargado de depósito, dashboard, y eventualmente notificaciones automáticas) deben enterarse sin que el modelo Producto conozca ni dependa de cada canal de aviso. Aplicación: Producto (o el servicio de stock) actúa como Subject y mantiene una lista de observadores que implementan la interfaz StockObserver; al notifyObservers() cada observador concreto (EncargadoDepositoObserver, DashboardObserver) decide cómo reaccionar.

3.5 Decorator — canales de notificación de alertas

Problema: una misma alerta de stock mínimo puede necesitar comportamientos adicionales combinables (registrar en un log, enviar un email) sin generar una subclase por cada combinación posible. Aplicación: una interfaz Alerta con una implementación base AlertaBase se envuelve mediante decoradores como LogAlertaDecorator y EmailAlertaDecorator, que agregan comportamiento antes o después de delegar en la alerta envuelta.

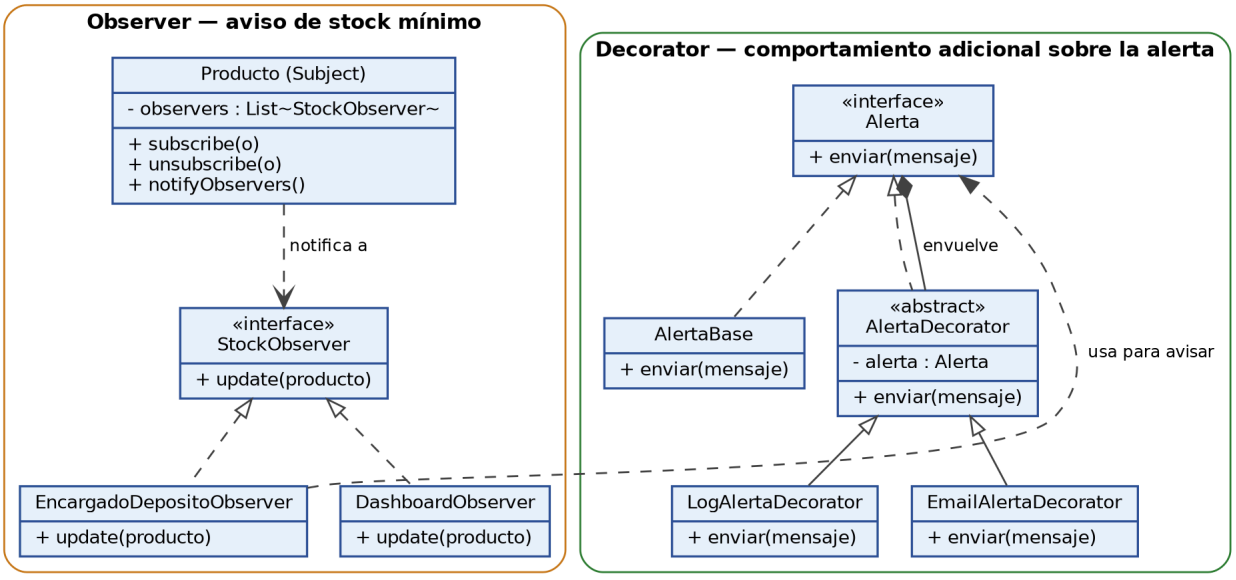


Figura 4. Observer para la notificación de alertas y Decorator para enriquecer su envío.

3.6 Adapter — exportación de reportes

Problema: entre las funcionalidades tentativas del sistema se contempla exportar reportes de movimientos e inventario en distintos formatos, cada uno con una librería externa de interfaz propia (por ejemplo, una librería para PDF y otra para Excel), lo que acoplaría la lógica de negocio a esas APIs concretas.

Aplicación: una interfaz común ReporteExporter es implementada por PdfReporteAdapter y ExcelReporteAdapter, que traducen el modelo interno de reportes a la interfaz particular de cada librería externa. El mismo enfoque podría reutilizarse a futuro para integrar sistemas de proveedores externos.

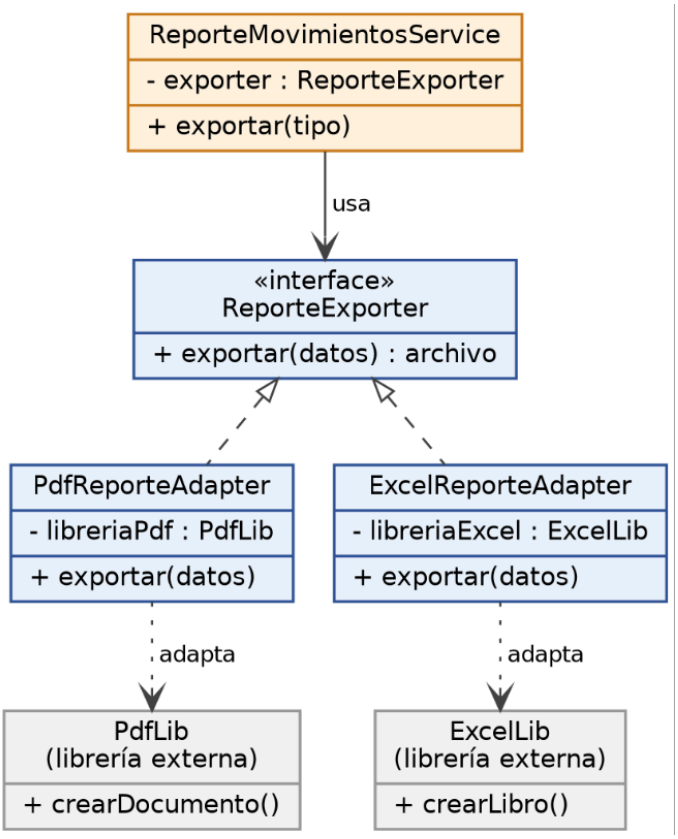


Figura 5. Adapter para desacoplar la exportación de reportes de las librerías externas.

3.7 Resumen de patrones aplicados

Patrón	Tipo	Se aplica en
Singleton	Creacional	Conexión/configuración a SQL Server
Factory Method	Creacional	Creación de MovimientoStock según su tipo
Strategy	Comportamiento	Validación y ejecución de cada tipo de movimiento
Observer	Comportamiento	Aviso de stock mínimo a distintos interesados
Decorator	Estructural	Combinación de canales de notificación de alertas
Adapter	Estructural	Exportación de reportes / integraciones externas

4. Conclusión

La estructura propuesta mantiene la separación de responsabilidades definida en la Entrega 1 y suma seis patrones de diseño concretos, cubriendo problemas de creación (Singleton, Factory Method), comportamiento (Strategy, Observer) y estructura (Decorator, Adapter). En conjunto, favorecen un diseño abierto a extensión y cerrado a modificación (Open/Closed), reducen el acoplamiento entre clases (Low Coupling) y concentran responsabilidades afines en cada componente (High Cohesion), preparando al sistema para incorporar en el futuro las funcionalidades tentativas (reportes, órdenes de compra, notificaciones) sin reescribir el núcleo del dominio.